

CENTRO UNIVERSITÁRIO DE BRASÍLIA (CEUB)

CURSO DE TECNOLOGIA

EM

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DISCIPLINA DE PROJETO INTEGRADOR I

José da Conceição Novais Neto

Arthur Amaral dos Santos

Matheus Covre Lacerda

Lucas Ferreira de Sousa

SUMARIO

1.	Introdução
1.1	Objetivo do Projeto
1.2	Escopo
1.3	Público-Alvo
2.	Requisitos do Sistema
2.1	Requisitos Funcionais
2.2	Requisitos Não Funcionais
3.	Arquitetura e Tecnologias
3.1	Diagrama de Arquitetura
3.2	Tecnologias Utilizadas
4.	Design e Interface do Usuário
4.1	Protótipo de Telas
4.2	Padrões de Design
5.	Modelagem do Sistema
5.1	Diagrama de Casos de Uso
5.2	Diagrama de Classes
5.3	Diagrama de Sequência
5.4	Diagrama Entidade-Relacionamento
6.	Desenvolvimento e Metodologia
6.1	Metodologia de Desenvolvimento
6.2	Estrutura do Código
7.	Gestão de Tarefas e <u>Kanban</u>
7.1	Uso do <u>Kanban</u>
7.2	Entrega do <u>Kanban</u>
8.	Testes e Validação
8.1	Estratégia de Testes
8.2	Ferramentas de Teste
9.	Implantação e Manutenção
9.1	Processo de Implantação
9.2	Plano de Manutenção
9.3	Versão Executável ou <u>Deploy</u> na Nuvem
9.4	Documentação Técnica
9.5	Manual do Usuário/Treinamento
10.	Conclusão

1. Introdução

1.1 Objetivo do Projeto

O projeto REGIIMENTOWEB tem como objetivo desenvolver uma plataforma digital responsiva para o grupo de pesquisa R.E.G.I.I.M.E.N.T.O., vinculado à Universidade de Brasília (UnB) e ao projeto INTERPARES TRUST AI. A plataforma busca centralizar informações institucionais, linhas de pesquisa, membros, produções científicas, eventos e dados analíticos em um único ambiente digital, facilitando o acesso da comunidade acadêmica e da sociedade à produção científica do grupo nas áreas de Ciência Arquivística Computacional (CAS), Arquitetura da Informação Multimodal (MIA), Processamento de Linguagem Natural, Engenharia de Ontologias, Multimodalidade e Deep Learning.

1.2 Escopo

O sistema compreende o desenvolvimento de um protótipo funcional de plataforma web, contemplando: portal institucional com histórico e apresentação do grupo; seção de linhas de pesquisa; repositório dinâmico de publicações acadêmicas (teses, dissertações e artigos); agenda de eventos; dashboards interativos com indicadores analíticos; e área de contato para interessados. O projeto não inclui, nesta fase, painel administrativo completo, controle avançado de usuários, integrações externas de grande escala nem área privada robusta. O MVP é desenvolvido com HTML, CSS, Tailwind CSS, JavaScript (ES6) e Chart.js, com dados gerenciados via arquivos JSON estáticos.

1.3 Público-Alvo

O sistema é voltado para três perfis principais de usuários: (1) Pesquisadores e docentes interessados nas linhas de pesquisa do grupo, que necessitam de acesso rápido a publicações e indicadores analíticos; (2) Estudantes de graduação e pósgraduação que desejam conhecer o grupo, suas áreas de atuação e formas de participação; e (3) Administradores de conteúdo responsáveis por manter as informações da plataforma atualizadas. A plataforma também se destina à comunidade acadêmica em geral e à sociedade interessada em ciência, tecnologia e educação digital.

2. Requisitos do Sistema

2.1 Requisitos Funcionais

RF-01 – Gestão de Conteúdo Estático: o sistema deve exibir o histórico, missão, visão e valores do grupo de forma clara. | RF-02 – Repositório Categorizado: a listagem de produções deve permitir busca por palavras-chave e filtros por tipo (Artigo, Tese, Workshop). | RF-03 – Dashboard Multimodal: o sistema deve renderizar

gráficos consumindo dados, representando a produção científica por ano e área. | RF-04 – Integração Lattes: cada perfil de integrante deve possuir um botão funcional de redirecionamento para o Currículo Lattes. | RF-05 – Formulário de Captação: área para interessados enviarem dados básicos (Nome, Email, Lattes) para contato futuro da coordenação.

2.2 Requisitos Não Funcionais

RNF-01 – Usabilidade: a interface deve garantir a navegação intuitiva com acesso à informação. | RNF-02 – Performance: o tempo de carregamento inicial (First Contentful Paint) não deve ultrapassar 2 segundos. | RNF-03 – Segurança: nenhum dado sensível de alunos (como CPF ou Matrícula) deve ser exposto no front-end ou em arquivos JSON públicos, em conformidade com a LGPD. | RNF-04 – Design Responsivo: layout adaptado utilizando Tailwind CSS.

3. Arquitetura e Tecnologias

3.1 Diagrama de Arquitetura

A arquitetura da solução é organizada em três camadas principais: (1) Front-end – responsável pela interface visual, navegação e apresentação dos conteúdos institucionais, acadêmicos e analíticos, implementado com HTML5, CSS3, Tailwind CSS e JavaScript (ES6); (2) Camada de Dados – gerenciada por arquivos JSON estáticos (publicacoes.json, eventos.json, membros.json, dados.json), que alimentam os componentes dinâmicos da interface sem necessidade de banco de dados nesta fase do MVP; (3) Visualização – biblioteca Chart.js responsável pela renderização de gráficos interativos (barras, linhas e rosca) nos dashboards analíticos. A hospedagem é realizada via Vercel ou GitHub Pages (tier gratuito), e o versionamento utiliza Git/GitHub.

3.2 Tecnologias Utilizadas

As tecnologias utilizadas no protótipo são: HTML5 e CSS3 para estruturação e estilização base; Tailwind CSS como framework CSS para design responsivo e identidade visual; JavaScript (ES6) para lógica de programação, manipulação de DOM e consumo de arquivos JSON; Chart.js para visualização de dados e renderização de gráficos interativos; AOS (Animate On Scroll) para animações de entrada de elementos; JSON para armazenamento e fornecimento de dados estáticos; Git e GitHub para versionamento e colaboração; Vercel/GitHub Pages para deploy e hospedagem gratuita.

4. Design e Interface do Usuário

4.1 Protótipo de Telas

O protótipo foi desenvolvido com as seguintes telas principais: (1) Home – página inicial com navegação clara, banner institucional e atalhos para as demais seções; (2) Sobre o Grupo – apresentação do histórico, missão e linhas de pesquisa do R.E.G.I.I.M.E.N.T.O.; (3) Equipe – cards dinâmicos carregados via JSON exibindo docentes e discentes; (4) Publicações – listagem dinâmica com filtros por tipo (Artigo, Tese, Workshop) e por ano, além de busca textual; (5) Eventos – agenda com distinção entre próximos eventos e eventos encerrados, dados via JSON; (6) Dashboards – painéis interativos com gráficos de produção científica por ano e por subárea; (7) Contato/Interessados – área com orientações de participação e formulário de captação. Capturas de tela do protótipo navegável estão disponíveis no repositório do projeto: <https://github.com/netonovais/Projeto-Integrador>

4.2 Padrões de Design

Os princípios de design adotados incluem: abordagem Mobile-First com layout responsivo via Tailwind CSS adaptado de 320px a 1920px; paleta de cores alinhada à identidade acadêmica (Azul Marinho UnB, Cinza Corporativo e Branco); tipografia Sans-serif para legibilidade; heurísticas de Nielsen como referência para usabilidade

(navegação intuitiva, feedback visual, consistência); conformidade com WCAG 2.1 Nível AA para acessibilidade; e carregamento dinâmico de conteúdo via JSON para separação entre apresentação e dados.

5. Modelagem do Sistema

5.1 Diagrama de Casos de Uso

Os principais casos de uso do sistema são: UC01 – Visualizar informações institucionais (ator: visitante); UC02 – Consultar linhas de pesquisa (ator: visitante/pesquisador); UC03 – Buscar e filtrar publicações acadêmicas (ator: pesquisador/estudante); UC04 – Visualizar agenda de eventos (ator: visitante); UC05 – Acessar dashboards analíticos (ator: visitante/parceiro institucional); UC06 – Enviar interesse de participação (ator: estudante/voluntário); UC07 – Acessar perfil Lattes de membros (ator: visitante/pesquisador). O diagrama de casos de uso completo está disponível no repositório GitHub do projeto.

5.2 Diagrama de Classes

As entidades principais modeladas no sistema são: Membro (id, nome, papel, lattes, linhasPesquisa[]); LinhaDePesquisa (id, nome, descricao); Publicacao (id, titulo, autor, tipo, ano, link); Evento (id, titulo, data, descricao, status); Contato (id, nome, email, lattes, mensagem); UsuarioAdministrador (id, nome, email, senha). Cada Membro está associado a uma ou mais LinhasDePesquisa. Publicacoes e Eventos são entidades independentes gerenciadas via arquivos JSON nesta fase do protótipo.

5.3 Diagrama de Sequência

O fluxo principal de acesso ao repositório de publicações ocorre da seguinte forma: (1) o usuário acessa a seção "Publicações" pela navegação principal; (2) o JavaScript (repositorio.js) realiza a leitura do arquivo publicacoes.json via Fetch API; (3) os dados são processados e renderizados dinamicamente como cards na interface; (4) o usuário aplica filtros por tipo ou ano, disparando a função de filtragem em JS; (5) os resultados filtrados são atualizados no DOM sem recarregamento de página. O mesmo padrão de sequência se aplica às seções de Eventos, Equipe e Dashboards.

5.4 Diagrama Entidade-Relacionamento

Na fase atual do MVP, os dados são gerenciados por arquivos JSON estáticos, sem banco de dados relacional. A estrutura lógica prevista para uma futura evolução inclui as tabelas: membros (id, nome, papel, email, lattes, id_linha_pesquisa); linhas_pesquisa (id, nome, descricao); publicacoes (id, titulo, autor, tipo, ano, link, id_linha_pesquisa); eventos (id, titulo, data, hora, local, descricao, status); contatos (id, nome, email, lattes, data_envio). Os relacionamentos principais são: membros N:M linhas_pesquisa; publicacoes N:1 linhas_pesquisa. O banco de dados sugerido para evolução futura é MySQL ou PostgreSQL.

6. Desenvolvimento e Metodologia

6.1 Metodologia de Desenvolvimento

O projeto adota metodologia ágil baseada em Scrum combinada com princípios de Design Thinking. As etapas seguidas foram: Empatia (levantamento das necessidades do grupo R.E.G.I.I.M.E.N.T.O.); Definição do problema (dispersão de informações científicas); Criação de personas (Pesquisador, Estudante e Administrador); Jornadas do usuário; Brainstorming de soluções; Definição de backlog e escopo do MVP. O desenvolvimento foi dividido em sprints com papéis definidos: Scrum Master (Matheus Covre), Product Owner (Arthur Silas), Arquiteto de Software (José Neto), Dev Front-end (Arthur Amaral) e AD/DBA (Lucas Ferreira). O planejamento foi organizado em 4 fases: Levantamento (2 semanas), Design e Estrutura (3 semanas), Desenvolvimento (5 semanas) e Entrega/Ajustes (2 semanas).

6.3 Repositório de Código-Fonte

Repositório GitHub: <https://github.com/netonovais/Projeto-Integrador>

6.2 Estrutura do Código

O código está organizado com separação clara entre estrutura (HTML), estilização (CSS/Tailwind) e lógica (JavaScript). Os arquivos JS seguem nomenclatura descritiva por funcionalidade: `repositorio.js` (publicações), `eventos.js` (agenda), `equipe.js` (membros), `dashboards.js` (gráficos). Os dados são centralizados em arquivos JSON na pasta `/data`. Boas práticas adotadas incluem: comentários explicativos nos blocos de lógica, funções nomeadas por responsabilidade, separação de concerns, uso de `const/let` (sem `var`) e ausência de dados sensíveis expostos no front-end.

O código segue boas práticas: funções comentadas com descrição de propósito, nomenclatura em `camelCase` para variáveis e funções, e estrutura de arquivos padronizada conforme convenção do projeto.

7. Gestão de Tarefas e Kanban

7.1 Uso do Kanban

O Kanban foi utilizado para organizar e acompanhar as tarefas do projeto de forma visual, com colunas: Backlog (itens identificados mas não iniciados), Em Andamento (tarefas em desenvolvimento na sprint atual), Em Revisão (itens concluídos aguardando validação) e Concluído (entregas finalizadas e aprovadas). As tarefas foram distribuídas conforme os papéis da equipe: PO priorizava o backlog, Scrum Master atualizava o quadro a cada reunião de sprint, e os demais membros moviam seus cards conforme o progresso real do desenvolvimento.

7.2 Entrega do Kanban

O quadro Kanban do projeto foi gerenciado via Trello. As capturas de tela do progresso das sprints, incluindo tarefas concluídas e pendentes, estão disponíveis no repositório: <https://github.com/netonovais/Projeto-Integrador>. As principais entregas registradas no Kanban incluem: identidade visual, arquitetura da informação, página inicial, seção institucional, linhas de pesquisa, repositório de publicações, agenda de eventos, área de interessados, dashboards analíticos e responsividade.

8. Testes e Validação

8.1 Estratégia de Testes

Foram realizados testes manuais no front-end, verificando navegação, renderização dinâmica, filtros e responsividade. A estratégia adotada envolveu testes funcionais (verificação de cada funcionalidade prevista no MVP), testes de interface (validação visual em diferentes resoluções de tela) e testes de aceitação (validação com o stakeholder do grupo R.E.G.I.I.M.E.N.T.O. em reunião realizada em 27/03/2026). Todos os 11 casos de teste definidos para o MVP foram executados com resultado OK.

8.2 Ferramentas de Teste

As ferramentas utilizadas nos testes foram: navegadores Google Chrome e Mozilla Firefox para validação cross-browser; DevTools do Chrome para simulação de dispositivos móveis e verificação de responsividade; extensão Lighthouse (Chrome) para análise de performance e acessibilidade; inspeção manual do DOM para verificar renderização dinâmica dos JSONs. Os testes foram documentados em tabela de casos de teste com colunas: ID, Funcionalidade, Procedimento, Resultado Esperado e Status.

Plano de Testes: os testes cobriram navegação principal, carregamento dinâmico via JSON (equipe, publicações e eventos), filtros por tipo e ano, busca textual, dashboards com Chart.js, responsividade mobile e área de contato.

Casos de Teste e Relatórios: 11 casos de teste foram executados (T01 a T11), cobrindo navegação, renderização dinâmica, filtros, busca, dashboards, hover nos gráficos, responsividade e contato. Todos com status OK. Relatório completo disponível no repositório GitHub.

9. Implantação e Manutenção

9.1 Processo de Implantação

O sistema será disponibilizado via hospedagem gratuita na Vercel ou Netlify. O processo de implantação ocorre por integração contínua com o repositório GitHub: a cada push na branch principal, o deploy é atualizado automaticamente. Não são necessárias configurações de servidor, pois a solução é totalmente estática (frontend + JSON). O acesso é realizado via URL pública gerada pela plataforma de hospedagem.

9.2 Plano de Manutenção

Futuras atualizações e correções serão gerenciadas via Git com uso de branches por funcionalidade (feature branches). A manutenção de conteúdo (publicações, eventos, membros) é realizada diretamente nos arquivos JSON, sem necessidade de acesso ao código-fonte principal. Correções de bugs seguirão o fluxo: identificação → abertura de issue no GitHub → desenvolvimento em branch → pull request → revisão → merge. A evolução do sistema (novas funcionalidades) será planejada em sprints futuras.

Plano de Suporte e Atualizações: manutenção corretiva será realizada mediante abertura de issues no GitHub. Manutenção evolutiva será planejada em sprints futuras, priorizando painel administrativo, integração com banco de dados e expansão dos dashboards.

Monitoramento e Logs: na fase atual do MVP, o monitoramento é realizado via painel da Vercel (logs de deploy e erros de build) e Google Search Console para análise de acesso. Em versões futuras, recomenda-se integração com ferramentas como Sentry (rastreamento de erros JS) e Google Analytics (métricas de uso).

9.3 Versão Executável ou Deploy na Nuvem

O protótipo navegável está disponível via Netlify. O repositório com o código-fonte completo encontra-se em: <https://github.com/netonovais/Projeto-Integrador>

9.4 Documentação Técnica

A documentação técnica para desenvolvedores futuros está disponível no README.md do repositório GitHub, contendo: estrutura de pastas, descrição dos arquivos JSON e seus campos, instruções de execução local, guia de contribuição e roadmap de funcionalidades previstas para versões futuras.

9.5 Manual do Usuário/Treinamento

O manual do usuário orienta a navegação pela plataforma: (1) acesse a página inicial e utilize o menu principal para navegar entre as seções; (2) em Publicações, utilize os filtros de tipo e ano ou a busca textual para localizar produções específicas; (3) em Eventos, verifique a distinção entre próximos eventos e encerrados; (4) em Dashboards, passe o mouse sobre os gráficos para visualizar valores detalhados; (5) em Contato/Interessados, preencha o formulário para demonstrar interesse em participar do grupo.

10. Conclusão

O projeto REGIIMENTOWEB cumpriu o objetivo proposto de desenvolver um MVP funcional para centralizar a produção científica e institucional do grupo R.E.G.I.I.M.E.N.T.O. A plataforma demonstrou viabilidade técnica e funcional, sendo aprovada pelo stakeholder na reunião de validação de 27/03/2026. Os principais benefícios entregues foram: centralização das informações do grupo em um único ambiente digital responsivo; visualização interativa de indicadores acadêmicos via dashboards com Chart.js; acesso facilitado ao repositório de publicações com filtros dinâmicos; e canal direto para engajamento de novos pesquisadores e voluntários. O protótipo serve como base sólida para evolução futura, incluindo integração com banco de dados, painel administrativo e expansão dos dashboards analíticos. A experiência com metodologia ágil (Scrum + Design Thinking) e a divisão de papéis bem definida contribuíram para a entrega dentro do prazo e escopo estabelecidos.

11 ANEXOS

Anexos disponíveis no github:

<https://github.com/netonovais/ProjetoIntegrador>